

# ANGULARJS - V

---

Sunita D. Rathod

# AngularJS Includes

- With AngularJS, you can include HTML from an external file.
- **ng-include** directive is used for this.

## Example

```
<body ng-app="">  
  
<div ng-include="'myFile.htm'"></div>  
  
</body>
```

# AngularJS Global API

- The Global API functions are accessed using the angular object.

| API                              | Description                               |
|----------------------------------|-------------------------------------------|
| <code>angular.lowercase()</code> | Converts a string to lowercase            |
| <code>angular.uppercase()</code> | Converts a string to uppercase            |
| <code>angular.isString()</code>  | Returns true if the reference is a string |
| <code>angular.isNumber()</code>  | Returns true if the reference is a number |

# Example

```
<div ng-app="myApp" ng-controller="myCtrl">
  <p>{{ x1 }}</p>
  <p>{{ x2 }}</p>
</div>
```

```
<script>
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
  $scope.x1 = "JOHN";
  $scope.x2 = angular.lowercase($scope.x1);
});
</script>
```

# AngularJS Animations

- An animation is when the transformation of an HTML element gives you an illusion of motion.
- To make your applications ready for animations, you must include the AngularJS Animate library:

```
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular-  
animate.js"></script>
```

# AngularJS Animations

- you must refer to the **ngAnimate** module in your application:

```
<body ng-app="ngAnimate">
```

- if your application has a name, add **ngAnimate** as a dependency in your application module:

```
<body ng-app="myApp">

<h1>Hide the DIV: <input type="checkbox" ng-model="myCheck"></h1>

<div ng-hide="myCheck"></div>

<script>
var app = angular.module('myApp', ['ngAnimate']);
</script>
```

# AngularJS Animations

- The **ngAnimate** module adds and removes classes.
- The directives in AngularJS who add/remove classes are:
  - **ng-show**
  - **ng-hide**
  - **ng-class**
  - **ng-view**
  - **ng-include**
  - **ng-repeat**
  - **ng-if**
  - **ng-switch**

# AngularJS Routing

- The **ngRoute** module helps your application to become a Single Page Application.
- If you want to navigate to different pages in your application, but you also want the application to be a SPA (Single Page Application), with no page reloading, you can use the **ngRoute** module.
- The **ngRoute** module *routes* your application to different pages without reloading the entire application.



# AngularJS Routing

- To make your applications ready for routing, you must include the AngularJS Route module:

```
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular-route.js"></script>
```

- Then you must add the **ngRoute** as a dependency in the application module:

```
var app = angular.module("myApp", ["ngRoute"]);
```

# AngularJS Routing

- Use the **\$routeProvider** to configure different routes in your application:

```
app.config(function($routeProvider) {  
  $routeProvider  
    .when("/", {  
      templateUrl : "main.htm"  
    })  
    .when("/red", {  
      templateUrl : "red.htm"  
    })  
    .when("/green", {  
      templateUrl : "green.htm"  
    })  
    .when("/blue", {  
      templateUrl : "blue.htm"  
    });  
});
```

# AngularJS Routing

- Your application needs a container to put the content provided by the routing.
- This container is the **ng-view** directive.

# AngularJS Routing

- There are **three** different ways to include the **ng-view** directive in your application:

```
<div ng-view></div>
```

```
<ng-view></ng-view>
```

```
<div class="ng-view"></div>
```

- Applications can only have **one ng-view** directive, and this will be the placeholder for all views provided by the route.

# Controllers

- With the **\$routeProvider** you can also define a controller for each "view".

```
var app = angular.module("myApp", ["ngRoute"]);
app.config(function($routeProvider) {
  $routeProvider
    .when("/", {
      templateUrl : "main.htm"
    })
    .when("/london", {
      templateUrl : "london.htm",
      controller : "londonCtrl"
    })
  });
```

```
    .when("/paris", {
      templateUrl : "paris.htm",
      controller : "parisCtrl"
    });
  });
app.controller("londonCtrl", function ($scope) {
  $scope.msg = "I love London";
});
app.controller("parisCtrl", function ($scope) {
  $scope.msg = "I love Paris";
});
```

# Template

- In the previous examples we have used the **templateUrl** property in the **\$routeProvider.when** method.
- You can also use the **template** property, which allows you to write HTML directly in the property value, and not refer to a page.

```
var app = angular.module("myApp", ["ngRoute"]);
app.config(function($routeProvider) {
  $routeProvider
    .when("/", {
      template : "<h1>Main</h1><p>Click on the links to change this content</p>"
    })
});
```

# The otherwise method

- In the previous examples we have used the **when** method of the **\$routeProvider**.
- You can also use the **otherwise** method, which is the default route when none of the others get a match.

```
.otherwise({  
  template : "<h1>None</h1><p>Nothing has been selected</p>"  
});
```

# AngularJS Dependency Injection

- AngularJS comes with a **built-in** dependency injection mechanism.
- It facilitates you to divide your application into multiple different types of components which can be injected into each other as dependencies.
- Dependency Injection is a software design pattern that specifies how components get holds of their dependencies. In this pattern, components are given their dependencies instead of coding them within the component.



# AngularJS Dependency Injection

- Modularizing your application makes it easier to reuse, configure and test the components in your application. Following are the core types of objects and components:
  - value
  - factory
  - service
  - provider
  - constant
- These objects and components can be injected into each other using AngularJS Dependency Injection.

# Value

- In AngularJS, value is a simple object. It can be a number, string or JavaScript object.
- It is used to pass values in factories, services or controllers during run and config phase.

```
//define a module  
var myModule = angular.module("myModule", []);  
  
//create a value object and pass it a data.  
myModule.value("numberValue", 100);  
myModule.value("stringValue", "abc");  
myModule.value("objectValue", { val1 : 123, val2 : "abc" } );
```

# Injecting a value

- To inject a value into AngularJS controller function, add a parameter with the same when the value is defined.

```
var myModule = angular.module("myModule", []);  
myModule.value("numberValue", 100);  
myModule.controller("MyController", function($scope, numberValue) {  
    console.log(numberValue);  
});
```

# Factory

- Factory is a function that is used to return value.
- When a service or controller needs a value injected from the factory, it creates the value on demand.
- It normally uses a factory function to calculate and return the value.

# Factory

```
var myModule = angular.module("myModule", []);  
myModule.factory("myFactory", function() {  
    return "a value";  
});  
  
myModule.controller("MyController", function($scope, myFactory) {  
    console.log(myFactory);  
});
```

# Service

- In AngularJS, service is a JavaScript object which contains a set of functions to perform certain tasks.
- Services are created by using `service()` function on a module and then injected into controllers.

```
//define a module  
var mainApp = angular.module("mainApp", []);  
  
...
```

# Service

```
//create a value object as "defaultInput" and pass it a data.  
mainApp.value("defaultInput", 5);  
  
//create a factory "MathService" which provides a method multiply to return multiplication of two numbers  
mainApp.factory('MathService', function() {  
    var factory = {};  
  
    factory.multiply = function(a, b) {  
        return a * b  
    }  
    return factory;  
});
```

# Service

```
//create a service which defines a method square to return square of a number.  
mainApp.service('CalcService', function(MathService){  
  this.square = function(a) {  
    return MathService.multiply(a,a);  
  }  
});  
  
//inject the service "CalcService" into the controller  
mainApp.controller('CalcController', function($scope, CalcService, defaultInput) {  
  $scope.number = defaultInput;  
  $scope.result = CalcService.square($scope.number);  
  $scope.square = function() {  
    $scope.result = CalcService.square($scope.number);  
  }  
});
```



# Provider

- In AngularJS, provider is used internally to create services, factory etc. during config phase (phase during which AngularJS bootstraps itself).
- It is the most flexible form of factory you can create.
- Provider is a special factory method with a `get()` function which is used to return the value/service/factory.

# Provider

```
//define a module
var mainApp = angular.module("mainApp", []);

...

//create a service using provider which defines a method square to return square
mainApp.config(function($provide) {
  $provide.provider('MathService', function() {
    this.$get = function() {
      var factory = {};
      factory.multiply = function(a, b) {
        return a * b;
      }
      return factory;
    };
  });
});
```

# Constants

- You cannot inject values into the `module.config()` function.
- Instead constants are used to pass values at config phase.

```
mainApp.constant("configParam", "constant value");
```

# Example to deploy all above

```
<!DOCTYPE html>
<html>
  <head>
    <title>AngularJS Dependency Injection</title>
  </head>
  <body>
    <h2>AngularJS Sample Application</h2>

    <div ng-app = "mainApp" ng-controller = "CalcController">
      <p>Enter a number: <input type = "number" ng-model = "number" />
    </p>
    <button ng-click = "square()">X<sup>2</sup> </button>
    <p>Result: {{result}}</p>
    </div>
```

```
<script src="http://ajax.googleapis.com/ajax/libs/angularjs/1.4.8/angular.min.js">
```

```
</script>
```

```
<script>
```

```
var mainApp = angular.module("mainApp", []);
```

```
mainApp.config(function($provide) {
```

```
  $provide.provider('MathService', function() {
```

```
    this.$get = function() {
```

```
      var factory = {};
```

```
      factory.multiply = function(a, b) {
```

```
        return a * b;
```

```
      }
```

```
      return factory;
```

```
    };
```

```
  });
```

```
});
```

```
mainApp.value("defaultInput", 10);
```

```
mainApp.factory('MathService', function() {
```

```
  var factory = {};
```

```
  factory.multiply = function(a, b) {
```

```
    return a * b;
```

```
  }
```

```
  return factory;
```

```
});
```

```
mainApp.service('CalcService', function(MathService){
```

```
  this.square = function(a) {
```

```
    return MathService.multiply(a,a);
```

```
  }
```

```
});
```

```
mainApp.controller('CalcController', function($scope, CalcService, defaultInput) {  
    $scope.number = defaultInput;  
    $scope.result = CalcService.square($scope.number);  
  
    $scope.square = function() {  
        $scope.result = CalcService.square($scope.number);  
    }  
});  
  
</script>  
  
    </body>  
  
</html>
```

# Output :

## AngularJS Sample Application

Enter a number:

$x^2$

Result: 100

## AngularJS Sample Application

Enter a number:

$x^2$

Result: 144



# AngularJS Built-in Helper Function

- **angular.copy**
- The angular.copy function takes an object, array or a value and creates a deep copy of it.
- Deep copy means that the new copy is assigned its own space in memory and changing will not updates the original.
- **Example :**

```
let obj = {name: "vespa", occupation: "princess"};
let cpy = angular.copy(obj);
cpy.name = "yogurt"
// obj = {name: "vespa", occupation: "princess"}
// cpy = {name: "yogurt", occupation: "princess"}
```

# AngularJS Built-in Helper Function

- `angular.equals`
- The `angular.equals` function compares and determines if 2 objects or values are equal, `angular.equals` performs a deep comparison and returns true if and only if at least 1 of the following conditions is met.

# AngularJS Built-in Helper Function

- **Syntax:**
- `angular.equals(value1, value2)`
  1. If the objects or values pass the `===` comparison
  2. If both objects or values are of the same type, and all of their properties are also equal by using `angular.equals`
  3. Both values are equal to NaN
  4. Both values represent the same regular expression's result.

# AngularJS Built-in Helper Function

## Examples

```
angular.equals(1, 1) // true
angular.equals(1, 2) // false
angular.equals({}, {}) // true, note that {}==={} is false
angular.equals({a: 1}, {a: 1}) // true
angular.equals({a: 1}, {a: 2}) // false
angular.equals(NaN, NaN) // true
```

# AngularJS Built-in Helper Function

- `angular.isArray`
- The `angular.isArray` function returns true if and only if the object or value passed to it is of the type Array.

## Examples

```
angular.isArray([]) // true  
angular.isArray([2, 3]) // true  
angular.isArray({}) // false  
angular.isArray(17) // false
```

# AngularJS Built-in Helper Function

- `angular.isDate`
- The `angular.isDate` function returns true if and only if the object passed to it is of the type `Date`.

## Examples

```
angular.isDate("lone star") // false  
angular.isDate(new Date()) // true
```

# AngularJS Built-in Helper Function

- **angular.merge**
- The function `angular.merge` takes all the enumerable properties from the source object to deeply extend the destination object.

## Examples

```
angular.merge({}, {}) // {}  
angular.merge({name: "king roland"}, {password: "12345"})  
// {name: "king roland", password: "12345"}  
angular.merge({a: 1}, [4, 5, 6]) // {0: 4, 1: 5, 2: 6, a: 1}  
angular.merge({a: 1}, {b: {c: {d: 2}}}) // {"a":1,"b":{"c":{"d":2}}}
```

# AngularJS Built-in Helper Function

- **angular.isDefined**
- The function `angular.isDefined` tests a value if it is defined.

## Examples

```
angular.isDefined(42) // true
angular.isDefined([1, 2]) // true
angular.isDefined(undefined) // false
angular.isDefined(null) // true
```



# AngularJS Built-in Helper Function

- **angular.isUndefined**
- The function angular.isUndefined tests if a value is undefined(it is effectively the opposite of angular.isDefined)

## Examples

```
angular.isUndefined(42) // false  
angular.isUndefined(undefined) // true
```

# AngularJS Built-in Helper Function

- `angular.toJson`
- The function `angular.toJson` will take an object and serialize it into a JSON formatted string. This function will remove all properties beginning with \$\$.
- As data needs to be serialized before passing through a network, this function is useful to turn any data you wish to transmit into JSON.

# AngularJS Built-in Helper Function

## Examples:

```
angular.toJson({name: "barf", occupation: "mog", $$somebizzareproperty: 42})  
// '{"name":"barf","occupation":"mog"}'  
angular.toJson(42)  
// "42"  
angular.toJson([1, "2", 3, "4"])  
// "[1,\"2\",3,\"4\"]"  
var fn = function(value) {return value}  
angular.toJson(fn)  
// undefined, functions have no representation in JSON
```

Thank you